Omówić koncepcję Aktywności i jej cyklu życia

Aktywność (Activity) to podstawowy komponent aplikacji w systemie Android, który reprezentuje interfejs użytkownika, czyli to, co widzi i czego używa użytkownik podczas korzystania z aplikacji. Aktywność może zawierać elementy interfejsu, takie jak przyciski, pola tekstowe, obrazy, listy, itp., a także logikę biznesową i funkcjonalność.

Cykl życia Aktywności to sekwencja zdarzeń, która zachodzi podczas jej tworzenia, uruchamiania, wznawiania, zatrzymywania, usuwania i przetwarzania różnych zdarzeń. Cykl życia Aktywności jest ważny, ponieważ ma wpływ na to, jak aplikacja działa i jak reaguje na różne zdarzenia, takie jak zmiana orientacji ekranu, wyjście z aplikacji czy zdarzenia systemowe.

Cykl życia Aktywności składa się z kilku etapów:

Tworzenie - w tym etapie tworzone są wszystkie zasoby potrzebne do działania Aktywności, takie jak widoki, obiekty i zmienne. Metoda onCreate() jest wywoływana tylko raz, podczas pierwszego tworzenia Aktywności.

Uruchamianie - w tym etapie Aktywność jest uruchamiana i staje się widoczna dla użytkownika. Metoda onStart() jest wywoływana, gdy Aktywność jest uruchamiana.

Wznawianie - w tym etapie Aktywność jest wznawiana po zatrzymaniu, ale nadal jest widoczna dla użytkownika. Metoda onResume() jest wywoływana, gdy Aktywność jest wznawiana.

Pauza - w tym etapie Aktywność jest zatrzymana, ale nadal jest widoczna dla użytkownika. Metoda onPause() jest wywoływana, gdy Aktywność jest zatrzymywana.

Zatrzymywanie - w tym etapie Aktywność jest zatrzymywana i przestaje być widoczna dla użytkownika. Metoda onStop() jest wywoływana, gdy Aktywność jest zatrzymywana.

Usuwanie - w tym etapie Aktywność jest usuwana i zwalniane są wszystkie zasoby, takie jak widoki, obiekty i zmienne. Metoda onDestroy() jest wywoływana, gdy Aktywność jest usuwana.

Podczas cyklu życia Aktywności, system Android wywołuje różne metody w odpowiednim czasie, w zależności od etapu, w którym znajduje się Aktywność. Dzięki temu programiści mogą kontrolować zachowanie Aktywności w różnych etapach cyklu życia, takie jak zapisywanie stanu, obsługa błędów, inicjalizacja i zwalnianie zasobów.

Omówić koncepcję intencji (Intent) i jej zastosowanie

Intent w Android Studio to obiekt, który reprezentuje zamiar, czyli intencję, aby przeprowadzić określoną akcję w aplikacji na platformie Android. Intencja ta może być wykorzystana do uruchomienia różnych komponentów aplikacji, takich jak aktywność, usługa lub odbiornik, a także do przekazywania danych pomiędzy nimi.
Zgodnie z dokumentacją Androida, Intenty są definiowane jako "abstrakcyjne opisy operacji, które mają zostać wykonane". Intenty określają akcję, która ma zostać wykonana, jakie dane mają zostać przesłane oraz jak aplikacja powinna reagować na zdarzenia związane z tą akcją. Intenty mogą przekazywać informacje takie jak nazwa akcji, kategoria, URI, typ MIME, klucz-wartość parametry i flagi.
Intenty stanowią podstawowy mechanizm komunikacji między różnymi komponentami aplikacji w Androidzie. Pozwalają one na tworzenie bardziej złożonych i interaktywnych aplikacji, ponieważ umożliwiają uruchamianie różnych komponentów aplikacji w odpowiedzi na różne zdarzenia, a także na przekazywanie danych pomiędzy nimi.

W Android Studio Intenty można tworzyć i konfigurować zarówno w kodzie Java, jak i Kotlin, a także wizualnie za pomocą edytora manifestu. Dzięki Intentom programiści mogą tworzyć aplikacje, które są bardziej elastyczne i dynamiczne, ponieważ pozwalają na interakcję między różnymi komponentami aplikacji.

Przykładem użycia Intentów w Android Studio może być uruchomienie nowej aktywności po naciśnięciu przycisku w istniejącej aktywności. W tym przypadku Intenty służą do przekazania informacji o akcji, jaką należy wykonać, a także do przekazania danych między aktywnościami.

Przykładowy kod w języku Java, który uruchamia nową aktywność po naciśnięciu przycisku, może wyglądać następująco:

```
Button button = findViewById(R.id.button);
button.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View v) {
        Intent intent = new Intent(MainActivity.this,
NewActivity.class);
        intent.putExtra("message", "Hello from MainActivity!");
        startActivity(intent);
    }
});
```

W powyższym kodzie po naciśnięciu przycisku zostanie utworzony nowy obiekt Intent, który określa aktywność, która ma zostać uruchomiona (NewActivity), oraz zawiera dodatkową informację w postaci wiadomości, która ma zostać przesłana do nowej aktywności. Następnie Intencja jest uruchamiana za pomocą metody startActivity(), co powoduje wyświetlenie nowej aktywności na ekranie urządzenia.

W nowej aktywności można następnie odebrać przekazane dane z Intencji w następujący sposób:

```
Intent intent = getIntent();
String message = intent.getStringExtra("message");
```

Omówić koncepcję fragmentów oraz grafu nawigacji

Fragmenty (Fragments) to kolejny ważny komponent systemu Android, który reprezentuje część interfejsu użytkownika, podobnie jak Aktywność, ale z tą różnicą, że Fragment może być wykorzystywany w ramach innej Aktywności. Fragmenty pozwalają na tworzenie modułowych i elastycznych interfejsów użytkownika, które mogą być łatwo modyfikowane i dostosowywane do różnych urządzeń.

Graf nawigacji (Navigation Graph) to narzędzie, które umożliwia programistom wizualizację i zarządzanie ścieżkami nawigacji w aplikacji Android. Graf nawigacji składa się z wierzchołków i krawędzi, które reprezentują Aktywności i Fragmenty oraz ścieżki, jakie użytkownik musi przejść, aby dotrzeć do konkretnego elementu interfejsu użytkownika.

Graf nawigacji ułatwia projektowanie i zarządzanie aplikacją, ponieważ pozwala na szybkie i łatwe dodawanie nowych elementów do interfejsu użytkownika i dostosowywanie ścieżek nawigacji do różnych potrzeb. Ponadto, graf nawigacji umożliwia łatwe zarządzanie stanem aplikacji i przejściami między poszczególnymi Aktywnościami i Fragmentami.

Programiści mogą tworzyć grafy nawigacji w Android Studio, korzystając z narzędzi takich jak NavHostFragment, NavController, NavGraph oraz NavDestination. Dzięki temu mogą projektować i zarządzać ścieżkami nawigacji w sposób prosty i intuicyjny.

Omówić metody tworzenie interfejsu użytkownika w aplikacjach mobilnych na system Android

W aplikacjach mobilnych na system Android istnieje kilka metod tworzenia interfejsu użytkownika. Wśród najpopularniejszych można wymienić:

XML Layouts - jest to najczęściej stosowana metoda do tworzenia interfejsu użytkownika w aplikacjach Android. Korzystając z XML Layouts, programista definiuje strukturę i wygląd interfejsu, używając dostępnych komponentów, takich jak przyciski, pola tekstowe, obrazy czy listy. Można też tworzyć niestandardowe widoki lub korzystać z gotowych bibliotek.

Java/Kotlin - tworzenie interfejsu użytkownika za pomocą kodu w językach Java lub Kotlin jest mniej popularne, ale wciąż stosowane, zwłaszcza w przypadku prostych interfejsów. Ta metoda polega na programowaniu komponentów interfejsu bezpośrednio w kodzie źródłowym aplikacji.

WYSIWYG Editors - WYSIWYG (What You See Is What You Get) Editors to narzędzia umożliwiające projektowanie interfejsu użytkownika za pomocą graficznego interfejsu. Narzędzia te pozwalają na umieszczanie i dostosowywanie komponentów interfejsu, a następnie wygenerowanie odpowiadającego kodu XML. Android Studio posiada wbudowany WYSIWYG Editor do tworzenia interfejsu użytkownika.

W każdej z tych metod programista korzysta z różnych narzędzi i technik, aby stworzyć interfejs użytkownika, który będzie działał poprawnie na urządzeniach mobilnych. Ważne jest, aby interfejs był intuicyjny i łatwy w obsłudze, a jego wygląd i struktura odpowiadały potrzebom użytkowników i celom aplikacji.

warstwa fizyczna sieci WLAN,

Warstwa fizyczna sieci WLAN jest to pierwsza warstwa w modelu referencyjnym ISO/OSI, odpowiadająca za transmisję danych w postaci sygnałów radiowych między urządzeniami sieciowymi. W sieciach WLAN, urządzenia komunikują się bezprzewodowo poprzez emisję fal radiowych, co wymaga zastosowania specjalnych urządzeń radiowych oraz anten.

Warstwa fizyczna określa między innymi:

sposoby kodowania i dekodowania danych,
sposób modulacji sygnału,
częstotliwość, na której są emitowane sygnały radiowe,
moc sygnału, która wpływa na zasięg sieci i jej prędkość,
technologie bezpieczeństwa, które umożliwiają szyfrowanie i deszyfrowanie danych.
Warstwa fizyczna sieci WLAN składa się z trzech podwarstw:

podwarstwa fizyczna (Physical Layer PLCP) - odpowiada za kodowanie i dekodowanie sygnałów oraz modulację/demodulację sygnału radiowego,
podwarstwa kontrolna dostępu do medium (Physical Layer Convergence Procedure PLCP) - odpowiada za przydzielanie dostępu do kanału transmisyjnego i sterowanie transmisją danych,
podwarstwa warstwy fizycznej (Service Access Point SAP) - odpowiada za łączenie warstwy fizycznej z warstwą łącza danych.

W sieciach WLAN najczęściej stosowane są standardy oparte na normie IEEE 802.11, takie jak np. 802.11a/b/g/n/ac/ax. Każdy z tych standardów definiuje m.in. zastosowaną modulację sygnału radiowego, pasmo częstotliwości, prędkość transmisji danych oraz sposoby szyfrowania danych.

Oto kilka dodatkowych informacji na temat warstwy fizycznej sieci WLAN:

Sposób modulacji sygnału: W zależności od zastosowanego standardu sieci WLAN, wykorzystywane są różne sposoby modulacji sygnału radiowego, np. modulacja amplitudy (AM), modulacja częstotliwości (FM), modulacja fazy (PSK) czy modulacja kąta (ASK). Współczesne standardy WLAN wykorzystują bardziej skomplikowane techniki modulacji, takie jak OFDM (Orthogonal Frequency Division Multiplexing) czy MU-MIMO (Multi-User Multiple Input Multiple Output).

Pasmo częstotliwości: Sieci WLAN mogą działać w różnych pasmach częstotliwości, takich jak 2,4 GHz lub 5 GHz. Sieci pracujące w paśmie 2,4 GHz są bardziej podatne na zakłócenia, ponieważ wiele innych urządzeń (np. kuchenki mikrofalowe, telefony bezprzewodowe) pracuje w tym samym paśmie. Sieci WLAN pracujące w paśmie 5 GHz oferują większą przepustowość i są mniej podatne na zakłócenia, ale mają mniejszy zasięg.

Prędkość transmisji danych: Prędkość transmisji danych w sieciach WLAN zależy od zastosowanego standardu, mocy sygnału oraz ilości zakłóceń. Najnowsze standardy WLAN, takie jak 802.11ax (również znany jako Wi-Fi 6), oferują przepustowość nawet do 9,6 Gb/s.

Technologie bezpieczeństwa: Aby zapewnić bezpieczeństwo transmisji danych w sieciach WLAN, stosuje się różne technologie, takie jak WEP (Wired Equivalent Privacy), WPA (Wi-Fi Protected Access) czy WPA2 (Wi-Fi Protected Access 2). WPA2 jest obecnie powszechnie stosowany i oferuje silne szyfrowanie danych oraz autoryzację użytkowników.

Anteny: Anteny wykorzystywane w sieciach WLAN wpływają na zasięg sieci oraz jakość sygnału. Dostępne są różne rodzaje anten, takie jak anteny kierunkowe, które skupiają sygnał w określonym kierunku, oraz anteny odbiorcze, które umożliwiają odbiór sygnału z większej odległości.

Warstwa fizyczna sieci WLAN jest niezbędna do zapewnienia bezprzewodowej transmisji danych między urządzeniami w sieci. Bez odpowiedniego działania tej warstwy, nie byłoby możliwe korzystanie z sieci WLAN.

sposoby reprezentacji liczb w komputerach (uzupełnienie do 1, do 2, reprezentacje zmiennoprzecinkowe),

Sposoby reprezentacji liczb w komputerach to kluczowe zagadnienie z dziedziny informatyki. W zależności od rodzaju reprezentacji, liczby mogą być przechowywane w różny sposób, co ma wpływ na dokładność, zakres, zużycie pamięci i szybkość obliczeń. Poniżej omówię trzy najpopularniejsze sposoby reprezentacji liczb w komputerach: uzupelnienie do 1, do 2 i reprezentacje zmiennoprzecinkowe.

Uzupełnienie do 1:
Uzupełnienie do 1 to sposób reprezentacji liczb całkowitych z użyciem bitów. Polega on na uzupełnieniu wszystkich bitów znaku po lewej stronie liczby (najstarszy bit) jedynkami, jeśli liczba jest ujemna, lub zerami, jeśli liczba jest nieujemna. Następnie całą liczbę zapisuje się binarnie.

Przykład: Liczbę -7 w systemie dziesiętnym można zapisać binarnie jako 11111001, gdyż najstarszy bit (1) oznacza, że liczba jest ujemna, a pozostałe bity to 7 w systemie dwójkowym.

Uzupełnienie do 2:
Uzupełnienie do 2 to bardziej popularna metoda reprezentacji liczb całkowitych w komputerach. Polega na uzupełnieniu wszystkich bitów znaku po lewej stronie jedynkami, jeśli liczba jest ujemna, lub wykonaniu operacji negacji bitowej i uzupełnieniu bitów znaku jedynkami, jeśli liczba jest nieujemna. Następnie całą liczbę zapisuje się binarnie.

Przykład: Liczbę -7 w systemie dziesiętnym można zapisać binarnie jako 11111001, gdyż najstarszy bit (1) oznacza, że liczba jest ujemna, a pozostałe bity to 7 w systemie dwójkowym. Natomiast uzupełnienie do 2 tej liczby wygląda następująco: 100010111, czyli wykonujemy negację bitową i dodajemy 1, aby otrzymać uzupełnienie do 2.

Reprezentacje zmiennoprzecinkowe:
Reprezentacje zmiennoprzecinkowe służą do reprezentowania liczb rzeczywistych, tj. zmiennych z częścią dziesiętną. Najpopularniejszym standardem reprezentacji zmiennoprzecinkowej jest standard IEEE 754. W tym standardzie liczba zmiennoprzecinkowa składa się z trzech części: znaku (1 bit), mantysy (zwykle 23 bity) i wykładnika (zwykle 8 bitów).

Przykład: Liczbę 3.14159 w standardzie IEEE 754 można zapisać jako: 010000000100
IEEE 754 to standard reprezentacji liczb zmiennoprzecinkowych w komputerach. Standard ten został opracowany przez Instytut Inżynierów Elektryków i Elektroników (IEEE) i określa sposób reprezentacji liczb zmiennoprzecinkowych w postaci binarnej.

Standard IEEE 754 określa sposób reprezentacji liczby zmiennoprzecinkowej za pomocą trzech elementów: bitu znaku, mantysy i wykładnika. W zależności od dokładności, z jaką chcemy przechowywać liczbę, w standardzie definiowane są różne formaty reprezentacji, takie jak float (32 bity), double (64 bity) i long double (80 lub 128 bitów).

Bit znaku określa, czy liczba jest dodatnia czy ujemna. Wartość 0 oznacza, że liczba jest dodatnia, a 1, że jest ujemna.

Mantysa to fragment liczby, który zawiera cyfry dziesiętne po przecinku i jest zapisana w postaci ułamka binarnego.

Wykładnik określa pozycję przecinka dziesiętnego liczby w ułamku binarnym.

Dzięki standardowi IEEE 754, liczby zmiennoprzecinkowe mogą być reprezentowane w sposób jednoznaczny i mogą być obliczane z dużą precyzją. Jednakże, w wyniku zaokrągleń czy też innych błędów numerycznych, w niektórych przypadkach obliczenia na liczbach zmiennoprzecinkowych mogą prowadzić do niedokładnych wyników.

mikroprocesory: obsługa przerwań

Mikroprocesory obsługują przerwania w celu reagowania na sygnały zewnętrzne, które wymagają natychmiastowej uwagi ze strony procesora. Przerwanie to sygnał, który jest wysyłany do procesora przez urządzenie zewnętrzne lub przez inny moduł wewnętrzny, który informuje procesor o tym, że wystąpiło zdarzenie, które wymaga natychmiastowej uwagi.

Aby obsłużyć przerwanie, mikroprocesor wykonuje następujące kroki:

Zapisuje stan procesora na stosie: Aby zapobiec utracie danych w momencie przerwania, mikroprocesor zapisuje zawartość rejestrów i flag procesora na stosie.

Ustawia flagę przerwania: Mikroprocesor ustawia specjalną flagę, która informuje go, że pojawiło się przerwanie.

Przekierowuje wykonanie do obsługi przerwania: Mikroprocesor przerywa wykonywanie bieżącej instrukcji i przechodzi do adresu, w którym znajduje się procedura obsługi przerwania.

Obsługuje przerwanie: W procedurze obsługi przerwania mikroprocesor przetwarza sygnał przerwania i wykonuje odpowiednie działania. Po zakończeniu obsługi przerwania, mikroprocesor usuwa zawartość stosu i wraca do wykonywania programu w miejscu, w którym zostało przerwane.

Obsługa przerwań jest kluczowa w wielu aplikacjach mikroprocesorowych, umożliwiając reakcję na zdarzenia zewnętrzne w czasie rzeczywistym, co jest niezbędne w wielu systemach wbudowanych i urządzeniach kontrolnych.

mikroprocesory: odmierzanie czasu